

NFL Fantasy Projections

Sofia Felan, Evan Leong, Adi Yarlagadda, Aanya Yataveli

The Problem

High Variance

A star player can score 0 pts one week and 35 the next. Even season-long averages poorly predict any single game.

Predicting Across Positions

QBs, RBs, WRs, and TEs score on completely different scales – a single model must generalize across all positions.

Multicollinearity

Passing yards, attempts, completions, and air yards are tightly correlated, making it hard to isolate true signal.

Injuries and Lineup Changes

Injuries happen quite frequently in the NFL. We have to measure player performance week by week since players can get benched or injured at any time.

Goal: Use machine learning to predict fantasy_points_standard for any NFL skill player on a week-by-week basis

Dataset description

NFL Stats 2021-2014 (Kaggle)

Raw Dataset

- **Instances:** 76287
- **Features:** 678
- Type: integer, float, object

Cleaned Dataset

- **Instances:** 46522
- **Features:** 852

- Offensive players only
- **Predictors:** player attributes and rolling average of usage metrics
- **Outcome variable:** fantasy points per player per match

	player_id	pfr_player_id	player_name	position	position_group	team	birth_date	height	weight	college_name	...	cum_first_down_rush	cum_third_down_converted
0	00-0038389	AbanIs00	Israel Abanikanda	RB	RB	NYJ	2002-10-05	70.0	216.0	Pittsburgh	...	1.0	0.0
1	00-0038389	AbanIs00	Israel Abanikanda	RB	RB	NYJ	2002-10-05	70.0	216.0	Pittsburgh	...	1.0	0.0
2	00-0038389	AbanIs00	Israel Abanikanda	RB	RB	NYJ	2002-10-05	70.0	216.0	Pittsburgh	...	1.0	0.0
3	00-0038389	AbanIs00	Israel Abanikanda	RB	RB	NYJ	2002-10-05	70.0	216.0	Pittsburgh	...	3.0	0.0
4	00-0038389	AbanIs00	Israel Abanikanda	RB	RB	NYJ	2002-10-05	70.0	216.0	Pittsburgh	...	3.0	0.0
5	00-0038389	AbanIs00	Israel Abanikanda	RB	RB	NYJ	2002-10-05	70.0	216.0	Pittsburgh	...	3.0	0.0
6	00-0031021	AbbrJa00	Jared Abbrederis	WR	WR	DET	1990-12-17	73.0	195.0	Wisconsin	...	0.0	1.0
7	00-0031021	AbbrJa00	Jared Abbrederis	WR	WR	DET	1990-12-17	73.0	195.0	Wisconsin	...	0.0	1.0
8	00-0031021	AbbrJa00	Jared Abbrederis	WR	WR	GB	1990-12-17	73.0	195.0	Wisconsin	...	0.0	0.0
9	00-0031021	AbbrJa00	Jared Abbrederis	WR	WR	GB	1990-12-17	73.0	195.0	Wisconsin	...	0.0	3.0

Data Cleaning

- Limited to numeric columns to facilitate predictions
- Removed columns that were mutations of other columns to prevent multicollinearity (eg "trend", "avg", "z-score", "cumulative")
- Removed leaky columns (fantasy_points_ppr and fantasy_points_half_ppr) because of correlation to outcome variable
- Removed unnecessary player information ("pfr_player_id" and "player_name")
 - We can access players by "player_id" primary key

Feature Engineering:

Rolling

```
windows = [3, 5, 8]
for col in rolling_cols:
    for w in windows:
        model_df[f'{col}_rolling{w}'] = (
            model_df.groupby('player_name')[col]
            .transform(lambda x: x.shift(1).rolling(window=w, min_periods=1).mean())
```

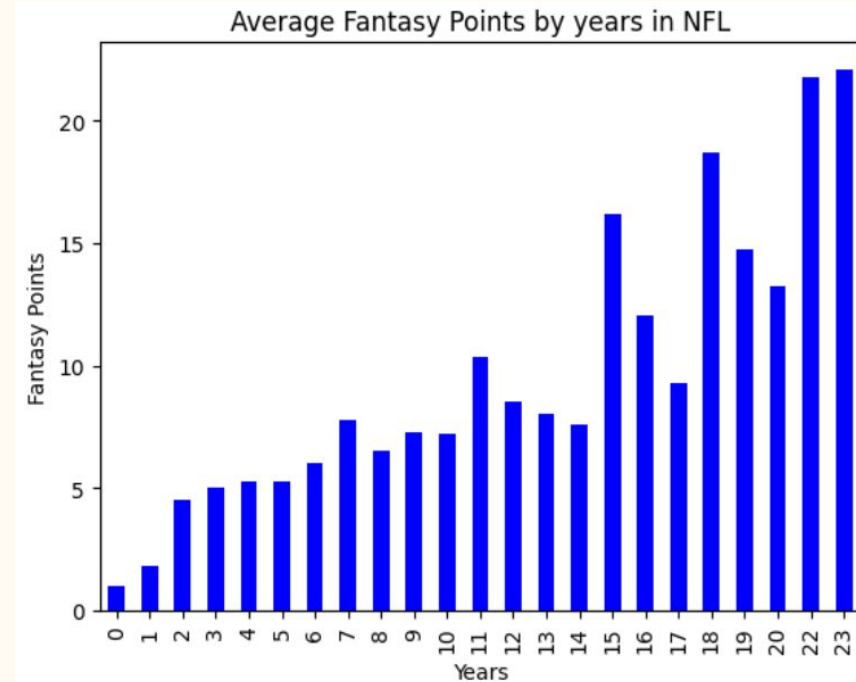
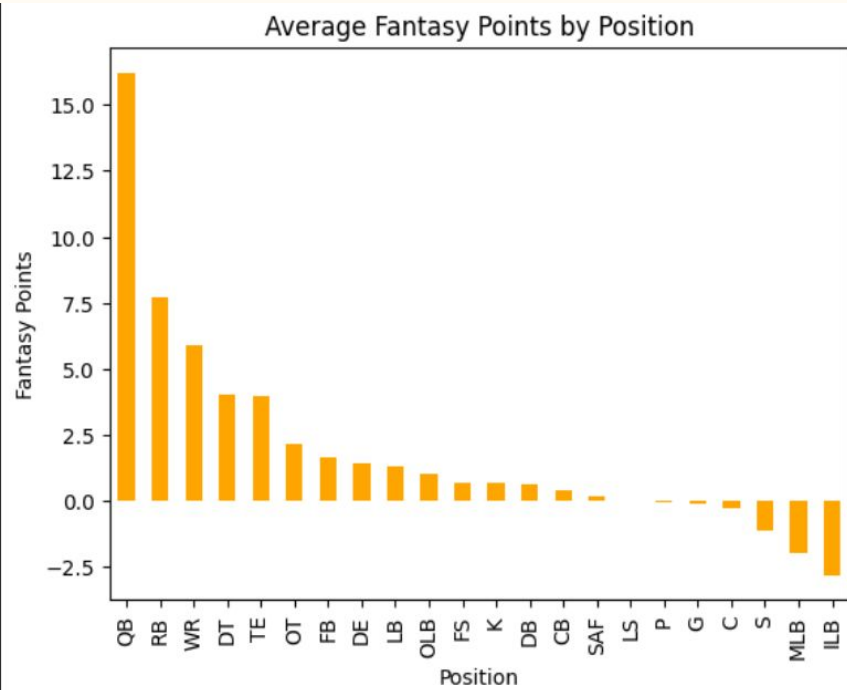
- Used past performance trends to better predict next week's fantasy points
- 3-game, 5-game, and 8-game rolling averages for key stats like yards, touchdowns, targets, touches, and fantasy points (numerical data_
- Smooths random weekly volatility and capture recent momentum (injuries, staffing changes, etc.)
- Used `.shift(1)` so only previous weeks were included
- Calculated separately for each player using `groupby(player_id)`

Feature Engineering:

SimpleImputer

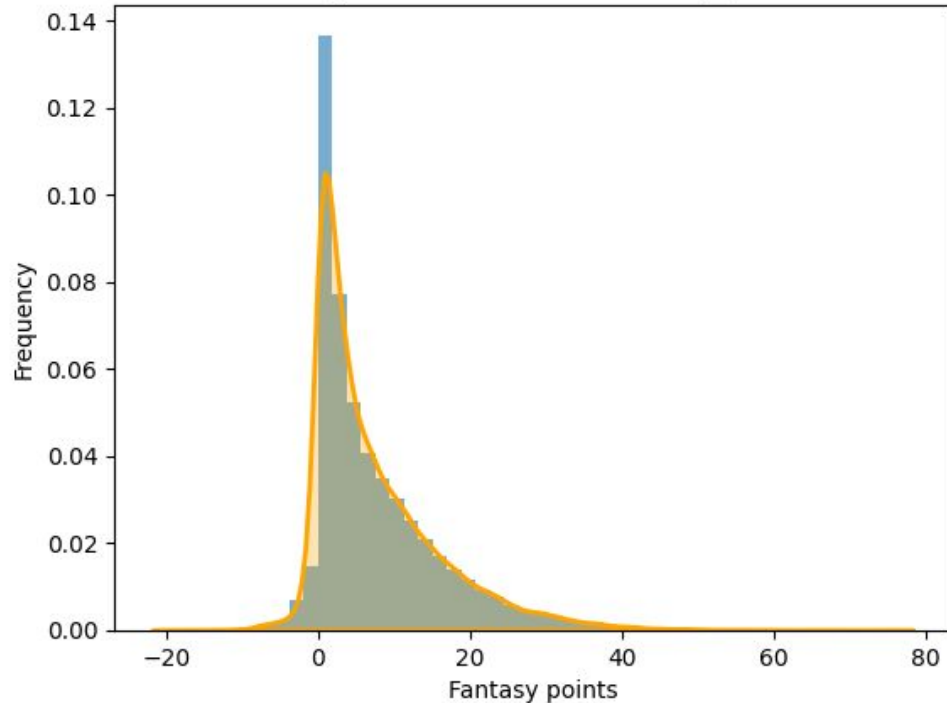
- Used SimpleImputer to fill missing values before modeling (often in a pipeline)
- Median imputation was applied to numeric features (more resistant to outliers than mean)
- Chose SimpleImputer over KNN imputation to improve computational efficiency
- If a categorical column had missing values, the missing entries were replaced with the most common category in that column
- After imputing missing categories, OneHotEncoder converted categorical variables into numeric dummy columns

EDA: Quarterbacks and veteran players are the most valuable

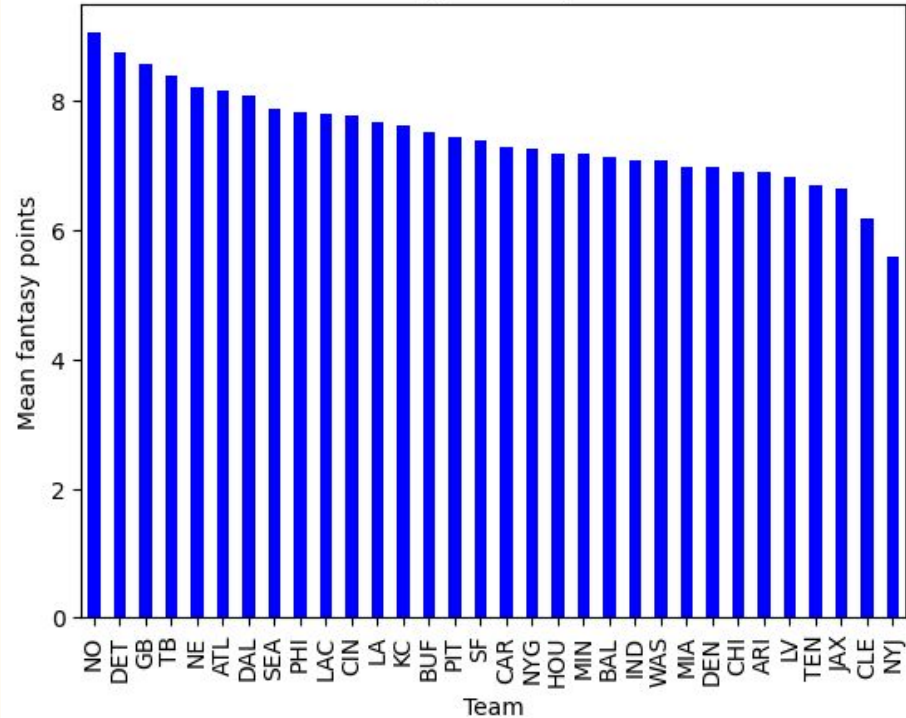


EDA: Most players get ~2 fantasy points, don't pick NYJ players for fantasy

Histogram distribution of fantasy points



Fantasy points by team



Feature Heatmap

KEY INSIGHTS

● Strong Cluster

Passing stats (yards, TDs, attempts) are tightly correlated – multicollinearity risk in models.

● Rushing Block

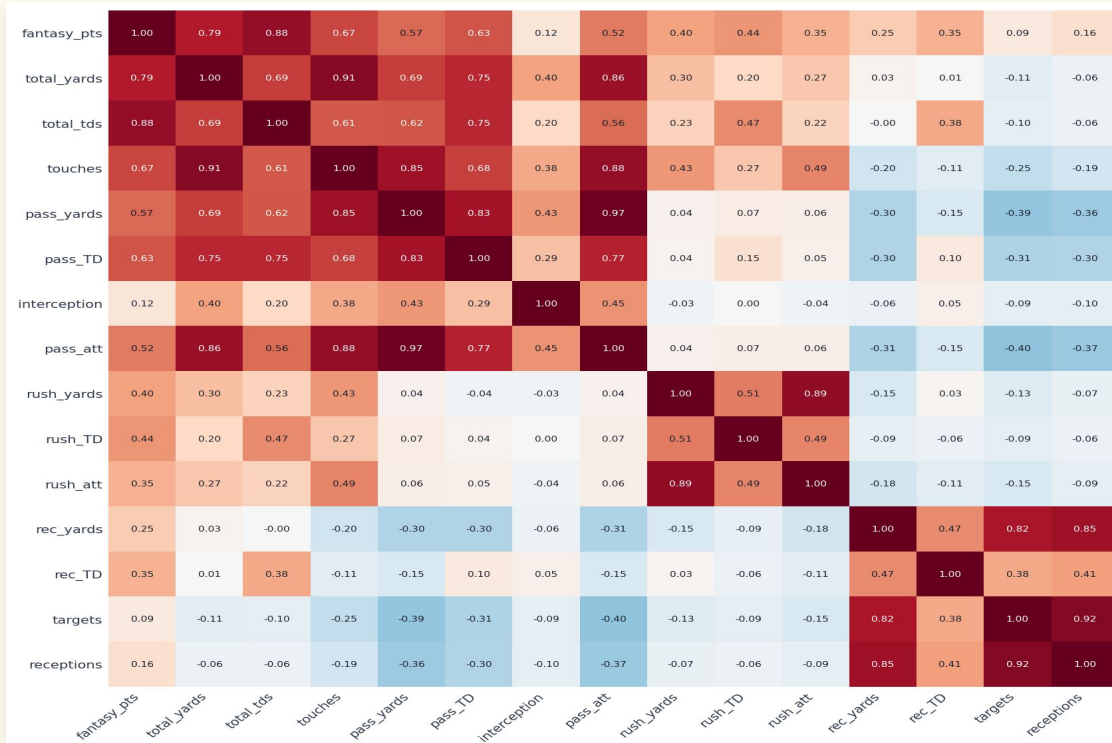
Rush yards, TDs & attempts form an independent cluster, largely separate from passing.

● Receiving Signal

Receiving yards, targets & receptions correlate strongly – good proxy features for WR value.

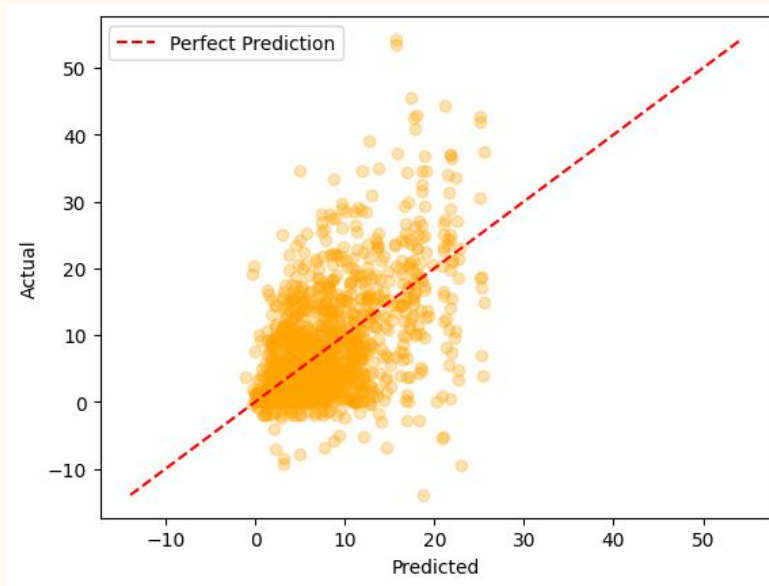
● Fantasy Output

fantasy_pts correlates broadly with yardage stats; weak link to interceptions (0.12).



Baseline Model

- Predicts weekly fantasy points based on rolling average
- Total average: 6.4795
- RMSE (train): 7.0242
- RMSE (test): 6.8613
- Not well-fit to perfect prediction line
- Predictions clustered around low-mid range of values, poor generalization ability on higher values



Bagging Model

Best bagging parameters:

- Bagging estimator max depth: 5
 - ◆ Max tree depth (too deep -> overfit, more shallow -> generalize better)
- Bagging Estimator Min Samples Leaf: 1
 - ◆ Min observations, prevents overly specific splits
- Bagging Max Samples: 0.7
 - ◆ Fraction of training rows used for each tree
- Bagging n Estimators: 100
 - ◆ # of trees in ensemble
- Want trees that are strong, different, and don't overfit

Bagging Train RMSE: 1.1455

Bagging Test RMSE: 1.2337

Random Forest: 3 Models

Baseline

Mean predictor

RMSE AVG

6.47

R^2 (Test)

-0.00

All features

Model 1

All Raw Box-Score Stats

RMSE TEST

1.94

RMSE TRAIN

1.54

41 features

Model 2

All Season & Rolling Avgs

RMSE TEST

2.64

RMSE TRAIN

2.03

48 features

Model 3

*Top-6 Combined Features
from Model 1 and 2*

RMSE TEST

2.33

RMSE TRAIN

2.10

6 features

Random Forest: 3 Models

Top 3 features from each prior model → 6 features, strong generalization

total_tds

imp: 0.672 ← Model 1

td_pct

imp: 0.163 ← Model 1

total_yards

imp: 0.143 ← Model 1

z_score_total_tds

imp: 0.01 ← Model 2

avg_total_tds

imp: 0.0028 ← Model 2

avg_total_yards

imp: 0.0025 ← Model 2

RMSE (Test)

2.33

Low error with only 6 features

R² (Test)

0.923

92.3% variance explained

Overfit Gap

0.016

Lowest of all models

★ Parsimonious model: near-Model-1 performance with 85% fewer features

Position-Specific Models

Training Model 3's 6 features separately for QB · RB · WR · TE

QB

RMSE (Test)

4.28

R² (Test)

0.862

Top Feature

total_tds (84%)

RMSE Train: 3.29

R² Train: 0.920

RB

RMSE (Test)

1.96

R² (Test)

0.939

Top Feature

td_pct (38%)

RMSE Train: 1.50

R² Train: 0.963

WR

RMSE (Test)

1.48

R² (Test)

0.946

Top Feature

total_tds (36%)

RMSE Train: 1.19

R² Train: 0.964

TE

RMSE (Test)

1.23

R² (Test)

0.937

Top Feature

total_tds (36%)

RMSE Train: 0.95

R² Train: 0.959

WR & TE achieved the lowest test RMSE — positional scoring is more predictable for receivers

Lasso Model

Best alpha: 0.01207

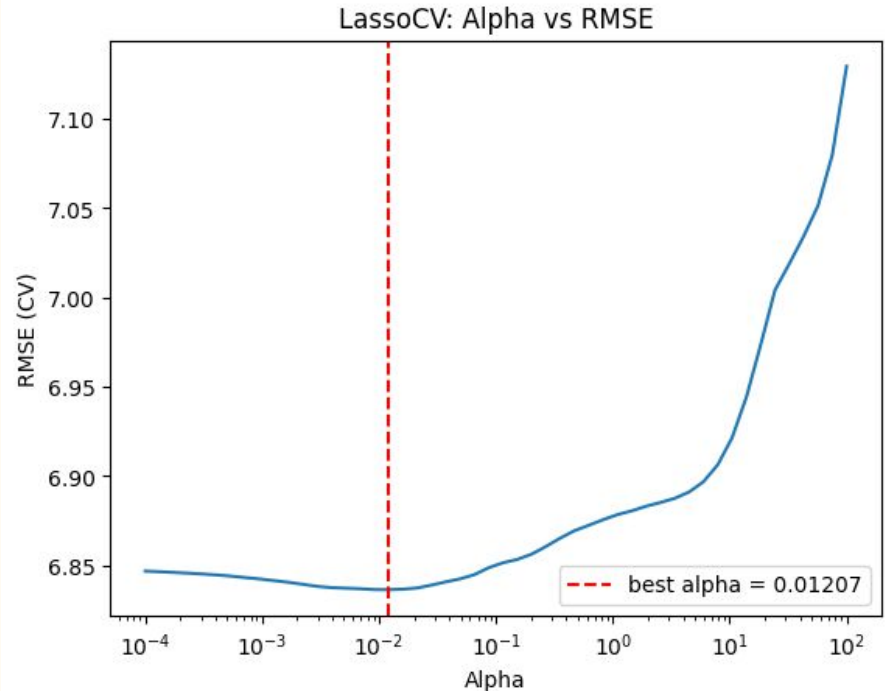
→ Calculated with 5-fold CV

Tuned to select the features with most weight → narrowed down from 852 to 79

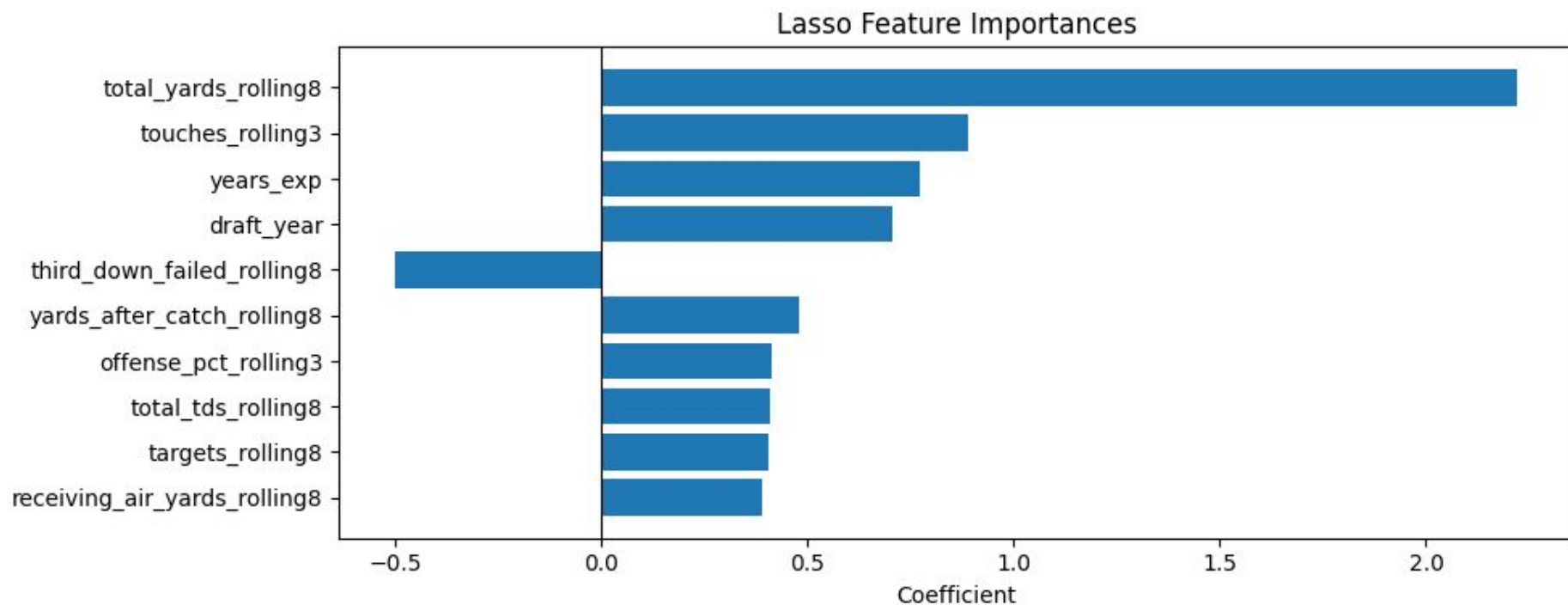
RMSE (train): 6.8137

RMSE (test): 6.6082

~3.69% improvement from baseline



Lasso Model



Lasso Model – Position-Specific

QB

Alpha: 0.0494

Lasso:

RMSE (train):
10.443

RMSE (test):
11.2594

Baseline:

RMSE (train):
10.8174

RMSE (test):
11.4477

RB

Alpha: 0.0869

Lasso:

RMSE (train):
7.0167

RMSE (test):
6.8443

Baseline:

RMSE (train):
7.3128

RMSE (test):
7.1696

WR

Alpha: 0.0655

Lasso:

RMSE (train):
5.7929

RMSE (test):
5.2552

Baseline:

RMSE (train):
5.9646

RMSE (test):
5.5676

TE

Alpha: 0.1526

Lasso:

RMSE (train):
4.3652

RMSE (test):
4.5177

Baseline:

RMSE (train):
4.4707

RMSE (test):
4.6108

Ridge Regression

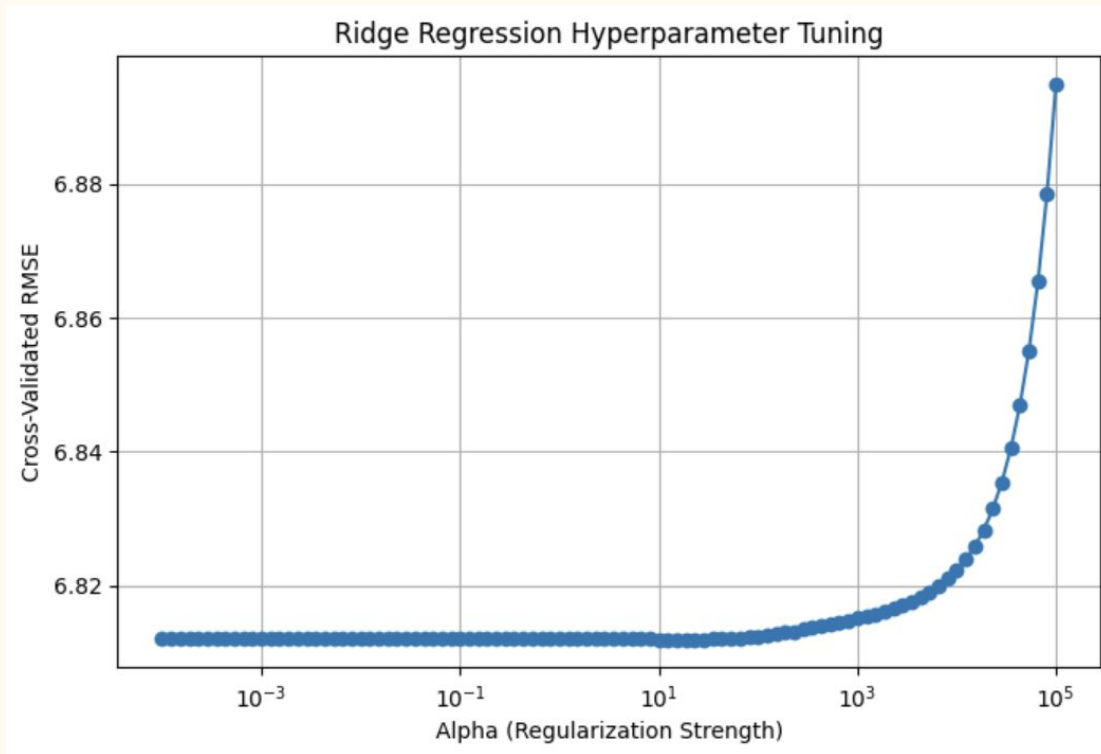
Ridge Insights:

- **Best alpha: 18.738**
 - Calculated with 5-fold CV
- **Baseline RMSE: 8.34**
- **Ridge RMSE: 6.72**
 - ~19% improvement over baseline
- **Ridge R^2 : 0.35**

Biggest features:

- Rolling averages (3, 5, 8 weeks)
- Player context (position, team, experience)

Ridge Model



Ridge Model – Position-Specific

QB

Ridge:

RMSE (test):
11.503

RMSE (train):
10.604

R^2 (test):
0.161

RB

Ridge:

RMSE (test):
7.121

RMSE (train):
6.980

R^2 (test):
0.204

WR

Ridge:

RMSE (test):
5.212

RMSE (train):
5.761

R^2 (test):
0.203

TE

Ridge:

RMSE (test):
4.529

RMSE (train):
4.312

R^2 (test):
0.123

GridSearchCV

- Works by training and evaluating a model on **every possible combination** of hyperparameters
- Used GridSearchCV to **test multiple hyperparameter** combinations for each model and identify the best performing settings
- Combined with **5-fold cross-validation**, allowing each parameter set to be evaluated on multiple train/test splits for more **reliable results**.

K-Fold Cross Validation

- Split data into **5 folds**
 - Strong tradeoff between model evaluation accuracy and computational efficiency
- Compared all models
- Evaluated models using **RMSE, MAE, and R^2**
- Sorted results by **lowest test RMSE** to identify the best-performing model

XGBoost: Position-Specific Models

QB

RMSE (Test)

4.08

R2 (Test)

0.875

RMSE Train

3.87

R2 Train: 0.889

0.8889

RB

RMSE (Test)

1.87

R2 (Test)

0.939

RMSE Train

1.79

R2 Train

0.943

WR

RMSE (Test)

1.31

R2 (Test)

0.950

RMSE Train

1.31

R2 Train

0.951

TE

RMSE (Test)

1.17

R2 (Test)

0.938

RMSE Train

1.09

R2 Train

0.941

WR and TE achieved the lowest test RMSE - consistent with RF position-specific results

Neural Network

Features

total_tds, td_pct, total_yards
z_score_total_tds, avg_total_tds,
avg_total_yards

Training

Optimizer: Adam (lr=0.001), standard learning
rate for NNs

Loss function: MSE

Epochs: 50

Loss: 33.2 converged to 5.38

2.62

RMSE (Test)

0.889

R2 (Test)

2.608

RMSE (Train)

0.890

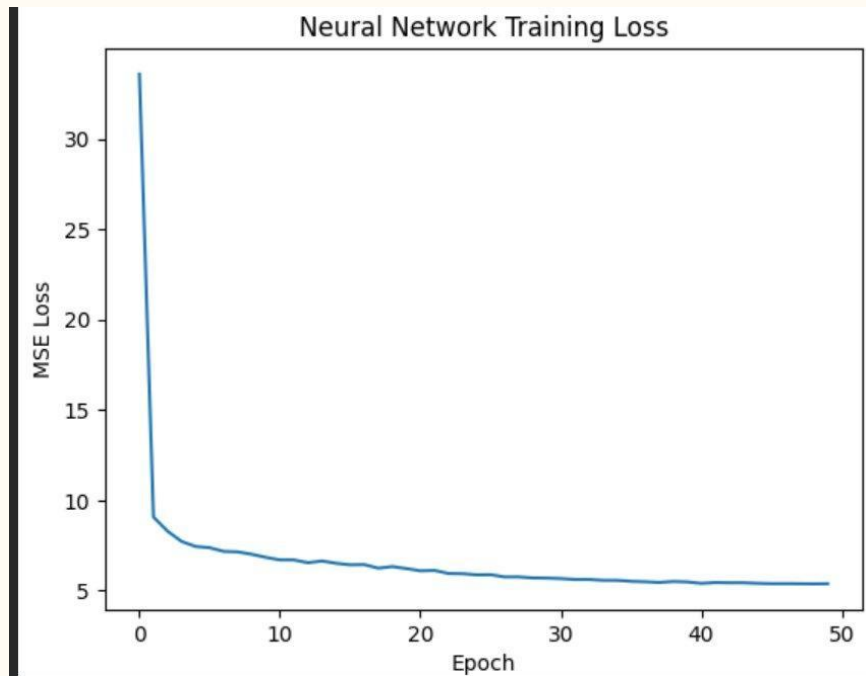
R2 (Train)

0.0011

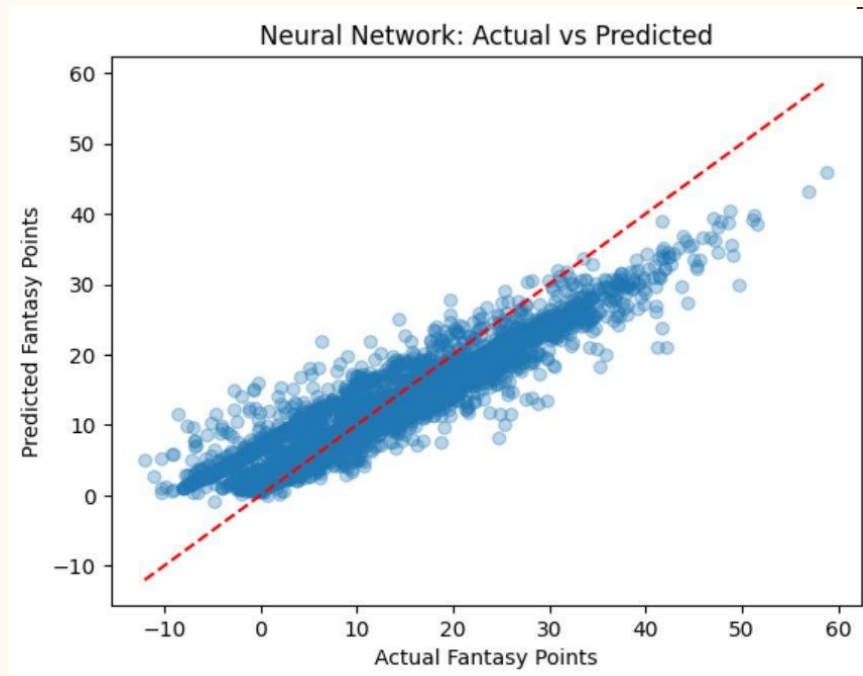
Overfit Gap

RF wins on accuracy; NN wins on generalization - dropout prevents overfitting on 6 features

Neural Network: Results



Training loss over 50 epochs



Actual vs. predicted fantasy points

Obstacles

- Overfitting models
 - Lasso
 - Random Forest
 - Neural Network
- Using future data to predict the future data -> overfitting
- Learning different patterns for different NFL positions
- Solutions:
 - Tuned hyperparameters to avoid overfitting
 - Cannot completely avoid overfitting

Conclusion

Implications

- Recent metrics are the most valuable for forecasting
- Ensemble methods (RF, Bagging, Boosting) outperform linear models
- Predictive analytics CAN be applied in this context

Limitations

- A lot of external data could improve accuracy (injury report, home vs away, etc)
- Results can vary by year, may need retraining each year (new rules, league conditions)

Applications

- Fantasy football enjoyers
 - Sports betting
 - Fantasy competitions
 - Analytical decision making and evaluation

THANK
YOU!
questions?